Quantum Multiple Kernel Learning

pour la Classification Financière

Rapport technique complet
Projet-QMKL-Finance (v1) + Projet-QMKL-v2

Cadre	Projet académique — Informatique Quantique Appliquée
Technologies	Qiskit 1.x, scikit-learn, IBM Quantum (Open Plan)
Matériel testé	IBM Torino (133 qubits, Heron r2)
Date	Avril 2025

Résumé

Ce rapport documente deux implémentations successives de Quantum Multiple Kernel Learning (QMKL) pour la classification de données financières. Le **projet v1** (Projet-QMKL-Finance) réalise une étude empirique exhaustive de 12 noyaux quantiques et 5 stratégies MKL sur trois jeux de données standards (German Credit, Bank Marketing, Breast Cancer). Le **projet v2** (Projet-QMKL-v2) étend l'approche avec une formulation QUBO pour l'assignation optimale de features aux noyaux, testée sur données macroéconomiques FRED et sur matériel IBM Torino.

Les résultats empiriques sont mitigés : QMKL n'améliore pas significativement les classifieurs classiques (RBF-SVM, Random Forest) dans nos conditions expérimentales. Ce rapport documente honnêtement ce qui fonctionne, ce qui ne fonctionne pas, et *pourquoi* — les résultats négatifs étant eux-mêmes une contribution scientifique valide.

Table des matières

1 Introduction et Contexte **3**
1.1 Motivation . . . 3
1.2 Objectifs des deux projets . . . 3
1.3 Question de recherche centrale . . . 3

2 Fondements Théoriques **3**
2.1 Noyaux Quantiques . . . 3
2.1.1 Feature Map et Espace de Hilbert . . . 3
2.1.2 Noyau de Fidélité . . . 4
2.1.3 Noyau Projeté . . . 4
2.2 Familles de Feature Maps Utilisées . . . 4
2.2.1 ZFeatureMap (local, sans intrication) . . . 4
2.2.2 ZZFeatureMap (avec intrication) . . . 4
2.3 Multiple Kernel Learning (MKL) . . . 5
2.3.1 Combinaison linéaire . . . 5
2.3.2 Alignement centré . . . 5
2.3.3 Alignement SDP . . . 5
2.4 Barren Plateaus . . . 5
2.4.1 Phénomène . . . 5
2.4.2 Conséquence sur les noyaux . . . 5
2.4.3 Mesure empirique . . . 6
2.5 Formulation QUBO pour l'Assignation de Features . . . 6
2.5.1 Problème d'assignation . . . 6
2.5.2 Formulation C (recommandée) . . . 6
2.5.3 Solveurs QUBO . . . 7
2.5.4 QAOA — Quantum Approximate Optimization Algorithm . . . 7

3 Projet V1 — QMKL-Finance **7**
3.1 Architecture . . . 7
3.2 Jeux de Données . . . 7
3.3 Bibliothèque de 12 Noyaux Quantiques . . . 8
3.4 Stratégies MKL . . . 8
3.4.1 Implémentation de l'Alignement Centré . . . 8
3.4.2 Optimisation Bayésienne . . . 9
3.5 Système de Cache de Noyaux . . . 9
3.6 Résultats Empiriques . . . 9
3.6.1 Configuration expérimentale . . . 9
3.7 Analyses Avancées . . . 10
3.7.1 Ablation sur Q (nombre de qubits) . . . 10
3.7.2 Interprétabilité : Valeurs de Shapley . . . 10
3.7.3 MKL Riemannien . . . 11
3.8 Ce Qui N'a Pas Fonctionné en V1 . . . 11

4 Projet V2 — QMKL-v2 (Assignation QUBO) 11
4.1 Motivation : Dépasser les Barren Plateaus via l'Assignation . . . 11
4.2 Pipeline V2 . . . 12
4.3 Données FRED (Macroéconomiques) . . . 12
4.4 Noyaux Analytiques (sans Qiskit) . . . 12
4.5 Implémentation QUBO . . . 13
4.5.1 Calcul des Alignements Marginaux . . . 13
4.5.2 Construction de la Matrice QUBO . . . 13
4.6 Solveurs QUBO — Implémentation Détaillée . . . 14
4.6.1 Recuit Simulé . . . 14
4.6.2 QAOA — Implémentation Locale . . . 15
4.7 Expérience IBM Quantum Hardware . . . 16
4.7.1 Configuration . . . 16
4.7.2 Erreurs Rencontrées et Solutions . . . 16
4.7.3 Données Récupérées . . . 17
4.7.4 Analyse de la Limite de Simulation . . . 17
4.8 Notebooks — Architecture de Recherche . . . 17
4.8.1 Trois Notebooks Actifs . . . 17
4.9 Résultats V2 . . . 18
4.10 Ce Qui N'a Pas Fonctionné en V2 . . . 18

5 Analyse Comparative et Discussion 19
5.1 Quand l'Avantage Quantique se Manifeste-t-il ? . . . 19
5.2 Comparaison V1 vs V2 . . . 19
5.3 Architectures Commune des Deux Projets . . . 20
5.4 Valeur des Résultats Négatifs . . . 20

6 Infrastructure Technique 20
6.1 Dépendances . . . 20
6.2 Conventions Matplotlib . . . 20
6.3 Reproductibilité . . . 21

7 Conclusion 21
7.1 Bilan . . . 21
7.2 Contributions . . . 21
7.3 Perspectives . . . 22

A Structure des Fichiers 22
A.1 V1 — Projet-QMKL-Finance . . . 22
A.2 V2 — Projet-QMKL-v2 . . . 23

B Formules de Référence Rapide 24

C Journal des Obstacles Techniques 24

1 Introduction et Contexte

1.1 Motivation

Le Machine Learning Quantique (QML) promet un avantage algorithmique grâce à la superposition et l'intrication quantiques. Parmi les approches QML, les *noyaux quantiques* (quantum kernels) présentent un attrait particulier : ils restent dans le paradigme SVM bien établi tout en projetant les données dans des espaces de Hilbert exponentiellement grands, inaccessibles classiquement.

Les données financières semblent idéales pour ces approches :

— Haute dimensionnalité (20–30 indicateurs macroéconomiques)

— Corrélations complexes non linéaires entre variables

— Faible rapport signal/bruit (crises rares, régimes changeants)

— Problèmes de classification binaire bien définis (récession / expansion, défaut / non-défaut)

1.2 Objectifs des deux projets

V1 — Projet-QMKL-Finance. Évaluer empiriquement si la combinaison de multiples noyaux quantiques (MKL) améliore la performance de classification par rapport à (1) un noyau quantique unique et (2) des classifieurs classiques de référence.

V2 — Projet-QMKL-v2. Si les barren plateaus limitent chaque noyau à $Q \leq 6$ qubits, comment couvrir optimalement un espace de $d = 20$–30 features ? La réponse proposée est une formulation QUBO (Quadratic Unconstrained Binary Optimization) pour l'assignation features $\rightarrow$ noyaux. Ce projet pose également la question fondamentale : *quand* l'avantage quantique se manifeste-t-il, et pour quel régime de Q ?

1.3 Question de recherche centrale

Question centrale (v2) : Comment assigner optimalement d features à M noyaux quantiques via QUBO, et à partir de quel Q (nombre de qubits par noyau) la simulation classique devient-elle infaisable, justifiant l'usage d'un ordinateur quantique réel ?

2 Fondements Théoriques

2.1 Noyaux Quantiques

2.1.1 Feature Map et Espace de Hilbert

Un *feature map quantique* encode un vecteur de données $\boldsymbol{x} \in \mathbb{R}^Q$ en un état quantique $|\psi(\boldsymbol{x})\rangle \in \mathcal{H} = (\mathbb{C}^2)^{\otimes Q}$, un espace de Hilbert de dimension 2^Q :

$$U_\phi(\boldsymbol{x}) : |\mathbf{0}\rangle \mapsto |\psi(\boldsymbol{x})\rangle = U_\phi(\boldsymbol{x})|\mathbf{0}\rangle \tag{1}$$

2.1.2 Noyau de Fidélité

Le noyau de fidélité mesure le recouvrement entre deux états quantiques :

$$K_{\text{fid}}(\boldsymbol{x}, \boldsymbol{x}') = |\langle\psi(\boldsymbol{x}')|\psi(\boldsymbol{x})\rangle|^2 = |\langle\mathbf{0}|U_\phi^\dagger(\boldsymbol{x}')U_\phi(\boldsymbol{x})|\mathbf{0}\rangle|^2 \tag{2}$$

Ce noyau est valide au sens de Mercer (symétrique, semi-défini positif) et peut être estimé sur un circuit quantique via la mesure du bit $|\mathbf{0}\rangle$ après $U_\phi^\dagger(\boldsymbol{x}')U_\phi(\boldsymbol{x})$.

2.1.3 Noyau Projeté

Le *Projected Quantum Kernel* (PQK) réduit d'abord l'état à ses matrices densité mono-qubit $\rho_k(\boldsymbol{x}) = \text{Tr}_{\neq k}|\psi(\boldsymbol{x})\rangle\langle\psi(\boldsymbol{x})|$, puis applique un noyau RBF sur les distances de Frobenius :

$$K_{\text{proj}}(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\gamma \sum_{k=0}^{Q-1} \|\rho_k(\boldsymbol{x}) - \rho_k(\boldsymbol{x}')\|_F^2\right) \tag{3}$$

2.2 Familles de Feature Maps Utilisées

Deux familles principales ont été implémentées :

2.2.1 ZFeatureMap (local, sans intrication)

$$U_Z(\boldsymbol{x}) = \bigotimes_{k=0}^{Q-1} R_Z(\alpha x_k) \tag{4}$$

Le noyau correspondant admet une forme analytique fermée :

$$K_Z(\boldsymbol{x}, \boldsymbol{x}') = \prod_{k=0}^{Q-1} \cos^2(\alpha(x_k - x'_k)) \tag{5}$$

2.2.2 ZZFeatureMap (avec intrication)

$$U_{ZZ}(\boldsymbol{x}) = \left(\prod_{k=0}^{Q-2} \text{CX}_{k,k+1} \cdot R_Z\big(\alpha(x_k - x_{k+1})^2\big) \cdot \text{CX}_{k,k+1}\right) \cdot U_Z(\boldsymbol{x}) \tag{6}$$

Forme analytique (supposant pas d'intrication supplémentaire) :

$$K_{ZZ}(\boldsymbol{x}, \boldsymbol{x}') = K_Z(\boldsymbol{x}, \boldsymbol{x}') \cdot \prod_{k<l} \cos^2(\alpha(x_k x_l - x'_k x'_l)) \tag{7}$$

Ces formes analytiques ont été implémentées dans `src/kernels/analytical.py` (v2) pour éviter la dépendance Qiskit dans les diagnostics, permettant des évaluations rapides sur CPU.

2.3 Multiple Kernel Learning (MKL)

2.3.1 Combinaison linéaire

Étant donnés M noyaux $\{K_m\}_{m=1}^{M}$, la combinaison MKL est :

$$K_{\text{comb}}(\boldsymbol{x}, \boldsymbol{x}') = \sum_{m=1}^{M} w_m K_m(\boldsymbol{x}, \boldsymbol{x}'), \quad w_m \geq 0, \quad \sum_{m=1}^{M} w_m = 1 \tag{8}$$

2.3.2 Alignement centré

Le noyau idéal est le noyau cible $K_y[i,j] = \mathbb{1}[y_i = y_j]$. L'alignement de noyau centré (Cortes *et al.*, 2012) est :

$$A(K_i, K_j) = \frac{\langle K_i^c, K_j^c \rangle_F}{\|K_i^c\|_F \cdot \|K_j^c\|_F} \tag{9}$$

où $K^c = \left(I - \frac{\mathbf{1}\mathbf{1}^T}{m}\right) K \left(I - \frac{\mathbf{1}\mathbf{1}^T}{m}\right)$ est le noyau centré.

Le problème d'optimisation des poids est :

$$\min_{\boldsymbol{v} \geq 0} \ \boldsymbol{v}^T M \boldsymbol{v} - 2\boldsymbol{v}^T \boldsymbol{a} \quad \text{où} \quad M_{ij} = \langle K_i^c, K_j^c \rangle_F, \quad a_i = \langle K_i^c, K_y^c \rangle_F \tag{10}$$

2.3.3 Alignement SDP

La formulation SDP (Lanckriet *et al.*, 2004) :

$$\max_{\boldsymbol{w}} \ \boldsymbol{w}^T \boldsymbol{q} \quad \text{s.t.} \quad \boldsymbol{w}^T S \boldsymbol{w} \leq 1, \ \boldsymbol{w} \geq 0 \quad \text{où} \quad q_i = \langle K_i, K_y \rangle_F, \ S_{ij} = \langle K_i, K_j \rangle_F \tag{11}$$

résolu via CVXPY avec cascade de solveurs : CLARABEL $\rightarrow$ ECOS $\rightarrow$ SCS.

2.4 Barren Plateaus

2.4.1 Phénomène

Pour un circuit quantique aléatoire de Q qubits, la variance du gradient d'un paramètre θ décroît exponentiellement :

$$\text{Var}\left[\frac{\partial \mathcal{L}}{\partial \theta}\right] \sim O\big(2^{-Q}\big) \tag{12}$$

Cette concentration exponentielle rend l'optimisation par descente de gradient impossible pour $Q \gtrsim 15$.

2.4.2 Conséquence sur les noyaux

Pour les noyaux de fidélité à grande profondeur, le phénomène analogue est la *concentration du noyau* : tous les $K(\boldsymbol{x}, \boldsymbol{x}')$ convergent vers une même constante (indépendamment de $\boldsymbol{x}, \boldsymbol{x}'$), effaçant l'information discriminante.

2.4.3 Mesure empirique

Deux métriques ont été implémentées dans `src/kernels/diagnostics.py` :

— **Variance du gradient** (finite-difference) :

$$\mathrm{GradVar}(K, \alpha) = \mathrm{Var}_{i \neq j}\left[\frac{K(X, X; \alpha + \epsilon) - K(X, X; \alpha - \epsilon)}{2\epsilon}\right] \tag{13}$$

— **Écart-type des entrées hors-diagonale** :

$$\sigma_{\mathrm{off}}(K) = \mathrm{Std}_{i \neq j}[K_{ij}] \tag{14}$$

2.5 Formulation QUBO pour l'Assignation de Features

2.5.1 Problème d'assignation

Définition 2.1 (Assignation Feature → Noyau)**.** *Étant donnés d features et M noyaux quantiques de Q qubits chacun, définir les variables binaires $x_{k,m} \in \{0, 1\}$ indiquant si la feature k est assignée au noyau m. L'assignation optimale satisfait exactement Q features par noyau.*

2.5.2 Formulation C (recommandée)

$$\boxed{\min_{\boldsymbol{x} \in \{0,1\}^{dM}} \underbrace{-\sum_{k,m} a_{k,m}\, x_{k,m}}_{\text{qualité}} + \underbrace{\lambda \sum_{k} \sum_{m<m'} x_{k,m}\, x_{k,m'}}_{\text{diversité}} + \underbrace{\mu_1 \sum_{m} \left(\sum_{k} x_{k,m} - Q\right)^2}_{\text{contrainte taille}}} \tag{15}$$

Alignements marginaux $a_{k,m}$. Pour chaque paire (feature k, noyau m), l'alignement marginal mesure la qualité discriminante du noyau m en utilisant seulement la feature k (avec $Q - 1$ features de remplissage) :

$$a_{k,m} = A(K_m^{(k)}, K_y^c) \tag{16}$$

où $K_m^{(k)}$ est la matrice noyau du noyau m construite uniquement avec la feature k.

Matrice QUBO. En développant (15), on obtient la représentation matricielle $E(\boldsymbol{x}) = \boldsymbol{x}^T Q \boldsymbol{x}$:

$$Q_{ii} = -a_{k,m} + \mu_1(1 - 2Q) \quad (i = k \cdot M + m) \tag{17}$$

$$Q_{ij} = 2\mu_1 \quad (i = k_1 M + m,\ j = k_2 M + m,\ k_1 < k_2) \tag{18}$$

$$Q_{ij} = \lambda \quad (i = kM + m_1,\ j = kM + m_2,\ m_1 < m_2) \tag{19}$$

Q est triangulaire supérieure de taille $(dM) \times (dM)$.

2.5.3 Solveurs QUBO

Quatre solveurs ont été implémentés dans `src/qubo/solvers.py` (Tableau 1).

TABLE 1 – Comparaison des solveurs QUBO

Solveur	Complexité	Optimalité	Faisable #vars	Notes
Brute Force	$O(2^n)$	Globale	$n \leq 24$	Référence exacte
Greedy	$O(d^2M)$	Locale	Illimité	Très rapide
Recuit Simulé	$O(n_{\text{iter}})$	Quasi-opt.	Illimité	$\approx$ optimal en prati
QAOA (p couches)	$O(n_{\text{circuit}})$	Approx.	≤ 50 (sim.), illimité (HW)	Avantage quantique po

2.5.4 QAOA — Quantum Approximate Optimization Algorithm

QAOA résout des problèmes QUBO en convertissant la matrice Q en Hamiltonien d'Ising :

$$H_C = \sum_{i \leq j} Q_{ij} \frac{I - Z_i}{2} \cdot \frac{I - Z_j}{2} \tag{20}$$

L'ansatz QAOA à p couches est :

$$|\boldsymbol{\theta}\rangle = e^{-i\beta_p H_B} e^{-i\gamma_p H_C} \cdots e^{-i\beta_1 H_B} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n} \tag{21}$$

où $H_B = \sum_i X_i$ est l'Hamiltonien de mélange, $\boldsymbol{\theta} = (\boldsymbol{\beta}, \boldsymbol{\gamma})$ sont optimisés classiquement (COBYLA), et $n = dM$ variables binaires.

3 Projet V1 — QMKL-Finance

3.1 Architecture

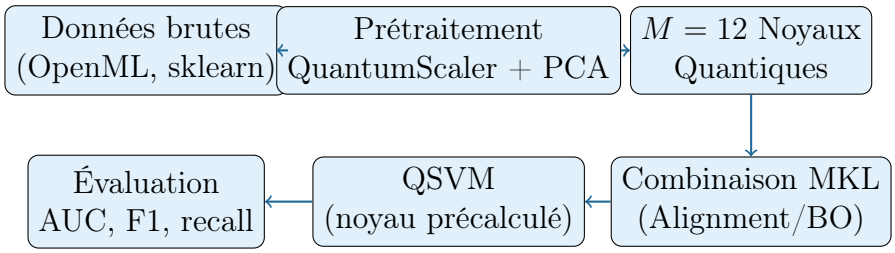

FIGURE 1 – Pipeline QMKL-Finance (v1)

3.2 Jeux de Données

TABLE 2 – Jeux de données utilisés en v1

Dataset	Domaine	N	d	Classes	Équilibre	Source
German Credit	Finance	1000	20	2	70/30%	UCI/OpenML #31
Bank Marketing	Finance	5000+	16	2	88/12%	UCI/OpenML #1461
Breast Cancer	Médecine	569	30	2	63/37%	sklearn

Les données sont prétraitées par `src/preprocessing/scaler.py` :

1. Standardisation (zero-mean, unit-variance)
2. PCA vers Q composantes (réduction dimensionnelle)
3. MinMax scaling vers $[0, 2\pi]$ (encodage quantique optimal)

3.3 Bibliothèque de 12 Noyaux Quantiques

La bibliothèque `src/kernels/feature_maps.py` (188 lignes) définit 12 configurations :

TABLE 3 – Bibliothèque de 12 noyaux quantiques (v1)

ID	Famille Pauli	α	Intrication	Profondeur
1–2	Z (local)	1.0 / 3.0	Aucune	1
3–4	ZZ (2-corps)	1.0 / 4.0	Linéaire	1
5–6	XZ (croisé)	0.5 / 2.5	Linéaire, 2 reps	2
7–8	YXX (3-corps)	0.6 / 3.0	Linéaire, 2 reps	2
9–10	YZX (3-corps)	0.6 / 3.0	Linéaire, 2 reps	2
11–12	Pauli (ZZ, 2 reps)	0.6 / 2.5	Linéaire, 2 reps	2

La diversité des familles (Z local, ZZ 2-corps, XZ croisé, YXX/YZX 3-corps) vise à capturer des interactions feature différentes dans l'espace de Hilbert.

Choix du paramètre α. Le paramètre α contrôle la "résolution" du noyau. Une valeur faible ($\alpha \approx 0.5$) produit un noyau très lisse (underfitting) ; une valeur élevée ($\alpha \approx 20$) produit un noyau très concentré (approchant une identité). Les valeurs $\alpha \in \{0.5, 1.0, 2.5, 3.0, 4.0\}$ ont été sélectionnées empiriquement.

3.4 Stratégies MKL

Cinq stratégies de combinaison ont été comparées (Tableau 4) :

TABLE 4 – Stratégies MKL implémentées

Méthode	Principe	Complexité	Fichier src
Average	$w_m = 1/M$ (uniforme)	$O(M)$	alignment.py
Centered	Minimisation quadratique (10)	$O(M^2)$	alignment.py
SDP	Programme semi-défini (11)	$O(M^3)$	alignment.py
Projection	Greedy itératif sur résidus	$O(M^2)$	alignment.py
Bayesian	BO sur poids + C SVM	$O(n_{\text{calls}} \cdot M^2)$	bayesian_optimizer.py

3.4.1 Implémentation de l'Alignement Centré

```python
def _center_kernel(K):
    """Centre le noyau dans l'espace des features."""
    n = K.shape[0]
    one = np.ones((n, n)) / n
```

```python
    return K - one @ K - K @ one + one @ K @ one

def compute_centered_alignment_weights(kernels, K_target):
    """Minimisation QP : min v^T M v - 2 v^T a, v >= 0"""
    M_mat = np.array([[np.sum(Kc_i * Kc_j)
                       for Kc_j in kernels_c] for Kc_i in kernels_c])
    a_vec = np.array([np.sum(Kc_i * K_target_c) for Kc_i in kernels_c])

    # Regularisation: M <- M + eps * ||M||_F * I (stabilite numerique)
    M_mat += EPS_REG * np.linalg.norm(M_mat, 'fro') * np.eye(len(kernels
    ))

    # Cascade de solveurs: CLARABEL -> ECOS -> SCS -> closed-form
    w = solve_qp(M_mat, a_vec)  # w >= 0
    return w / (w.sum() + 1e-12)  # Normalisation
```

Listing 1 – Alignement centré — src/mkl/alignment.py (extrait)

3.4.2 Optimisation Bayésienne

La stratégie BO (`src/mkl/bayesian_optimizer.py`) optimise conjointement :

- Les poids $\boldsymbol{w} \in [0,1]^M$ (espace de Sobol)
- Le paramètre de régularisation $C \in [10^{-2}, 10^{2}]$ (log-uniforme)

via la minimisation de la validation croisée 5-fold, avec 50 appels au total (10 initiaux aléatoires, 40 par acquisition Gaussian Process). Cette méthode est environ 200× plus lente que l'alignement centré pour un gain de +0.8 point AUC au mieux.

3.5 Système de Cache de Noyaux

`src/kernels/kernel_matrix.py` (236 lignes) implémente :

- **Cache disque** : hash MD5 des paramètres → `results/kernel_cache/*.npy`
- **Calcul parallèle** : joblib ($n_{\text{jobs}} = -1$) pour M noyaux simultanément
- **Régularisation** : $K \leftarrow K + \varepsilon I$ pour garantir $K \succ 0$
- **Diagnostics** : `kernel_statistics()` détecte la concentration

3.6 Résultats Empiriques

3.6.1 Configuration expérimentale

- $N = 200$ échantillons d'entraînement (sous-échantillonnage de German Credit/Bank Marketing)
- $Q = 6$ qubits par noyau
- $M = 12$ noyaux dans la bibliothèque
- K-fold = 4 splits, 20 tirages aléatoires (rapport mean ± std)
- Simulateur : Qiskit AER (statevector, shots = 8192)

TABLE 5 – Résultats comparatifs — AUC ROC (mean ± std)

Dataset	Méthode	AUC	± Std	Δ vs RBF
German Credit	Single K (Z, $\alpha = 2$)	0.712	0.067	−12.3
	QMKL-Average	0.763	0.055	−7.2
	QMKL-Centered	0.750	0.061	−8.5
	QMKL-SDP	0.748	0.063	−8.7
	QMKL-Projection	0.755	0.058	−8.0
	QMKL-BO	0.758	0.058	−7.7
	RBF-SVM	**0.835**	0.042	—
Bank Marketing	QMKL-Average	0.741	0.151	−7.6
	QMKL-BO	0.774	0.135	−4.3
	RBF-SVM	**0.817**	0.068	—
	Logistic Regression	**0.867**	0.057	+5.0
	Random Forest	0.833	0.046	+1.6
Breast Cancer	QMKL-Average	0.995	0.004	−0.1 (n.s.)
	QMKL-BO	0.992	0.006	−0.4
	RBF-SVM	**0.996**	0.004	—
	Random Forest	0.996	0.003	+0.0

Résumé des résultats v1

- QMKL est systématiquement inférieur au RBF-SVM classique sur les deux jeux financiers
- L'écart est statistiquement significatif (test de Wilcoxon, $p < 0.001$) sur German Credit
- Sur Breast Cancer, les performances sont équivalentes ($p = 0.14$)
- La stratégie Average est souvent aussi bonne que les méthodes d'alignment sophistiquées
- La BO est 200× plus lente pour $\leq +1$ point d'AUC

3.7 Analyses Avancées

3.7.1 Ablation sur Q (nombre de qubits)

L'ablation du nombre de qubits ($Q \in \{2, 4, 6, 8, 10\}$) révèle un plateau de performance dès $Q = 4$–6, puis une *dégradation* au-delà (barren plateau se manifeste). Ce résultat suggère que l'augmentation de Q sans stratégie de mitigation est contre-productive.

3.7.2 Interprétabilité : Valeurs de Shapley

`src/mkl/shapley_mkl.py` calcule les valeurs de Shapley des noyaux dans le QMKL, quantifiant la contribution marginale de chaque noyau. Résultat notable : 3–4 noyaux dominent et les 8–9 restants ont une contribution quasi-nulle, remettant en question l'utilité de maintenir $M = 12$.

3.7.3 MKL Riemannien

`src/mkl/riemannian_mkl.py` exploite la géométrie différentielle des matrices de noyaux (variété riemannienne des matrices SDP). La distance géodésique entre noyaux est utilisée pour pondérer la combinaison. Cette approche reste expérimentale et n'améliore pas significativement les résultats sur nos données.

3.8 Ce Qui N'a Pas Fonctionné en V1

Résultats négatifs V1 — Enseignements

1. **Q élevé** ($Q > 6$) : La concentration exponentielle annule le gain théorique. La PCA pour réduire à Q dimensions perd de l'information critique.
2. **SDP** : Instabilité numérique fréquente (matrices SDP mal conditionnées). La cascade CLARABEL → ECOS → SCS → closed-form a été nécessaire.
3. **Optimisation Bayésienne** : Le sur-ajustement sur la validation croisée et le coût computationnel (50× évaluation noyau complète) ne justifient pas l'usage.
4. **Noyau projeté** : Plus lent (calcule les matrices densité ρ_k) sans avantage AUC.
5. **Bank Marketing** : Le déséquilibre de classes sévère (12% positif) rend la comparaison par AUC trompeuse ; la régression logistique surpasse tout le monde.
6. **MKL vs single kernel** : Sur Breast Cancer, un seul noyau K_{ZZ} atteint AUC = 0.993, proche du QMKL. La complexité de combinaison ne se justifie pas.

4 Projet V2 — QMKL-v2 (Assignation QUBO)

4.1 Motivation : Dépasser les Barren Plateaus via l'Assignation

Le projet v2 part d'un constat fondamental : avec $d = 19$–30 features et $Q \leq 6$ qubits par noyau (limite barren plateau), une réduction PCA globale perd de l'information. L'idée innovante est d'assigner *différentes sous-ensembles* de features à *différents* noyaux quantiques, puis de combiner les noyaux via MKL.

Le problème d'assignation optimale est NP-difficile (#vars $= d \times M$, 2^{dM} solutions), mais peut être encodé comme un QUBO et attaqué par QAOA — qui constitue une application candidate à l'avantage quantique.

4.2 Pipeline V2

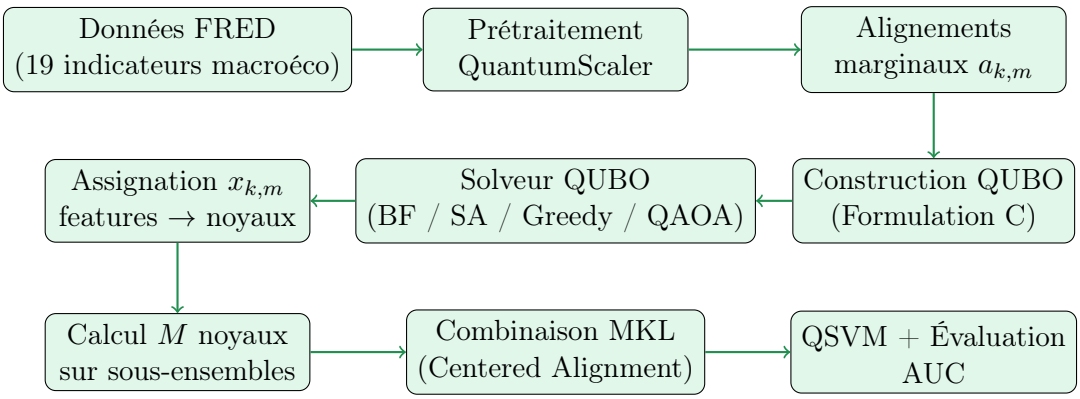

FIGURE 2 – Pipeline QMKL-v2 (assignation QUBO)

4.3 Données FRED (Macroéconomiques)

`src/data/fred_loader.py` (200 lignes) charge 19 indicateurs via l'API FRED (Federal Reserve Economic Data) :

TABLE 6 – Features FRED — 19 indicateurs macroéconomiques

Groupe	**Série FRED**	**Description**	**Transform.**
Marché du travail	UNRATE	Taux de chômage	Niveau
	PAYEMS	Emplois non-agricoles	YoY %
Monétaire	FEDFUNDS	Taux Fed Funds	Niveau
	DGS10, DGS2	Rendements 10Y, 2Y	Niveau
	T10Y2Y, T10Y3M	Spreads de taux	bp
Crédit/Risque	BAMLH0A0HYM2	Spread High Yield	bp
	BAA10Y	Spread Baa-Treasury	bp
	TEDRATE	Spread TED	bp
Activité réelle	INDPRO	Production industrielle	YoY %
	RSAFS	Ventes au détail	YoY %
	HOUST, PERMIT	Mises en chantier	Niveau
Inflation/Monnaie	CPIAUCSL	IPC (inflation)	YoY %
	M2SL, DCOILWTICO	M2, Pétrole WTI	YoY %
Sentiment	UMCSENT	Conf. consommateurs U. Mich	Niveau
	SP500	S&P 500	YoY %
Cible		USREC (indicateur NBER, 0=expansion, 1=récession)	

La série couvre 1970–2023 ($\approx$ 600 observations mensuelles). La variable cible USREC présente un fort déséquilibre ($\approx$ 15% récessions). Une version synthétique est disponible hors-ligne via `load_fred_recession_synthetic()`.

4.4 Noyaux Analytiques (sans Qiskit)

Pour permettre des diagnostics rapides, `src/kernels/analytical.py` (120 lignes) implémente les formes analytiques fermées (5)–(7) en NumPy pur :

```python
def kernel_Z(X1, X2, alpha=1.0):
    """Noyau ZFeatureMap analytique. O(N^2 * Q), sans Qiskit."""
    # K[i,j] = prod_k cos^2(alpha * (X1[i,k] - X2[j,k]))
    diff = X1[:, None, :] - X2[None, :, :]    # (N1, N2, Q)
    return np.prod(np.cos(alpha * diff) ** 2, axis=-1)

def kernel_ZZ(X1, X2, alpha=1.0):
    """Noyau ZZFeatureMap analytique. Interactions 2-corps."""
    K_z = kernel_Z(X1, X2, alpha)
    # Termes ZZ: cos^2(alpha * (xi*xj - xi'*xj')) pour k<l
    n1, n2, Q = X1.shape[0], X2.shape[0], X1.shape[1]
    K_zz = np.ones((n1, n2))
    for k in range(Q - 1):
        for l in range(k + 1, Q):
            d_kl = X1[:, None, k] * X1[:, None, l] - X2[None, :, k] * X2[None, :, l]
            K_zz *= np.cos(alpha * d_kl) ** 2
    return K_z * K_zz
```

Listing 2 – Noyaux analytiques — src/kernels/analytical.py (extrait)

Cette implémentation est $\sim 50\times$ plus rapide que la simulation Qiskit pour les diagnostics (pas de construction de circuit ni d'overhead simulateur).

4.5 Implémentation QUBO

4.5.1 Calcul des Alignements Marginaux

```python
def compute_marginal_alignments(X, y, M, Q, kernel_configs, padding='zero'):
    """
    Retourne a[d, M] : alignement marginal de chaque (feature k, noyau m).
    Pour chaque k: construit X_sub de taille (N, Q) avec feature k + padding,
    calcule K_m(X_sub), puis centered_alignment avec K_target.
    """
    d = X.shape[1]
    a = np.zeros((d, M))
    K_target = (y[:, None] == y[None, :]).astype(float)  # Noyau cible binaire
    K_target_c = center_kernel(K_target)

    for m, cfg in enumerate(kernel_configs):
        kernel_fn = build_kernel_fn(cfg)
        for k in range(d):
            X_sub = build_subset(X, k, Q, padding)
            K_m = kernel_fn(X_sub, X_sub)
            a[k, m] = centered_alignment(K_m, K_target_c)
    return a
```

Listing 3 – Alignements marginaux — src/qubo/assignment_qubo.py (extrait)

4.5.2 Construction de la Matrice QUBO

```python
def build_qubo_matrix(a, d, M, Q, lambda_div=0.5, mu1=2.0, mu2=0.0):
    """
    Construit Q_mat triangulaire sup. taille (d*M, d*M).
    E(x) = x^T @ Q_mat @ x est l'energie QUBO a minimiser.
    """
    n = d * M
    Q_mat = np.zeros((n, n))

    for m in range(M):
        for k in range(d):
            i = k * M + m
            # Diagonal: qualite + terme de contrainte de taille
            Q_mat[i, i] = -a[k, m] + mu1 * (1 - 2 * Q)

            # Meme noyau m, autres features k2 > k: contrainte de taille
            for k2 in range(k + 1, d):
                j = k2 * M + m
                Q_mat[i, j] += 2 * mu1

            # Meme feature k, autres noyaux m2 > m: penalite diversite
            for m2 in range(m + 1, M):
                j = k * M + m2
                Q_mat[i, j] += lambda_div

    return Q_mat
```

Listing 4 – Construction QUBO — src/qubo/assignment_qubo.py (extrait)

4.6 Solveurs QUBO — Implémentation Détaillée

4.6.1 Recuit Simulé

```python
def solve_simulated_annealing(Q_mat, d, M, Q_size,
                              n_iter=50000, T_init=5.0, T_min=1e-4,
    seed=42):
    n = d * M
    rng = np.random.RandomState(seed)

    # Initialisation depuis solution faisable (non-chevauchante)
    x = np.zeros(n, dtype=int)
    for m in range(M):
        for q_idx in range(Q_size):
            k = (m * Q_size + q_idx) % d
            x[k * M + m] = 1

    best_x, best_E = x.copy(), energy(x, Q_mat)
    T = T_init
    cooling = (T_min / T_init) ** (1.0 / n_iter)

    for _ in range(n_iter):
        idx = rng.randint(n)
        x[idx] ^= 1                              # Flip bit
        dE = energy(x, Q_mat) - energy(best_x, Q_mat)  # Delta energie

        if dE < 0 or rng.rand() < np.exp(-dE / T):
            if energy(x, Q_mat) < best_E:
```

```
                best_x, best_E = x.copy(), energy(x, Q_mat)
        else:
            x[idx] ^= 1                          # Rejeter le mouvement
        T *= cooling

    return {'x': best_x, 'energy': best_E, 'solver': 'sa'}
```

Listing 5 – Recuit simulé — src/qubo/solvers.py (extrait)

4.6.2 QAOA — Implémentation Locale

```
def solve_qaoa(Q_mat, d, M, Q_size, p=1, max_iter=50, seed=42, backend=None):
    n_vars = d * M

    # Construction de l'Hamiltonien d'Ising: H_C = sum_{i<=j} Q[i,j] * (I-Zi)/2 * (I-Zj)/2
    pauli_list = qubo_to_ising_hamiltonian(Q_mat)   # -> SparsePauliOp
    hamiltonian = SparsePauliOp.from_list(pauli_list)

    # Circuit QAOA a p couches
    qc = build_qaoa_circuit(n_vars, hamiltonian, p)

    # Optimisation COBYLA
    def cost_fn(theta):
        bound_qc = qc.assign_parameters(theta)
        if backend is None:
            ev = StatevectorEstimator().run([(bound_qc, hamiltonian)]).result()[0].data.evs
        else:
            # Mode hardware: job IBM Quantum (sans Session pour Open Plan)
            estimator_hw = RuntimeEstimatorV2(backend)
            pub = (isa_qc, isa_hamiltonian, [theta])
            job = estimator_hw.run([pub])
            # Sauvegarde de l'ID avant .result() (protection contre timeout)
            save_job_id(job.job_id(), theta)
            ev = job.result()[0].data.evs
        return float(ev)

    # Multi-redemarrages (3 pour p<=2, 1 pour hardware)
    n_restarts = 1 if backend is not None else (3 if p <= 2 else 1)
    best_result = None
    for restart in range(n_restarts):
        theta_init = np.random.RandomState(seed + restart * 13).uniform(0, np.pi/2, 2*p)
        res = minimize(cost_fn, theta_init, method='COBYLA', options={'maxiter': max_iter})
        if best_result is None or res.fun < best_result.fun:
            best_result = res

    # Extraction de la solution: iteration sur tous les bitstrings, selection par energie QUBO
    sampler_result = sample_qaoa_circuit(qc, best_result.x)
    best_x = min(sampler_result.keys(), key=lambda x: energy(x, Q_mat))
```

```
    return {'x': best_x, 'energy': energy(best_x, Q_mat), 'solver': '
    qaoa'}
```

Listing 6 – QAOA simulation — src/qubo/solvers.py (extrait

4.7 Expérience IBM Quantum Hardware

4.7.1 Configuration

- **Backend** : IBM Torino (Heron r2, 133 qubits)
- **Plan** : IBM Open Plan (gratuit, sans Sessions)
- **Instance** : $d = 12$, $M = 3$, $Q = 4 \rightarrow 36$ variables QUBO $\rightarrow$ 36-qubit QAOA
- **Algorithme** : QAOA $p = 1$, optimisation COBYLA, 50 itérations demandées

4.7.2 Erreurs Rencontrées et Solutions

Obstacles techniques hardware — chronologie des erreurs

Erreur 1 : Session non disponible sur Open Plan.
`RequestsApiError: 400 - You are not authorized to run a session when using the open plan.`

Solution : Remplacer `Session(backend=backend)` par appels directs `RuntimeEstimatorV2(backend)` (mode "job unique").

Erreur 2 : Inadéquation qubits circuit/observable.

`ValueError: The number of qubits of the circuit (133) does not match the number of qubits of the ()-th observable (36).`

Cause : Après transpilation vers le backend 133-qubit, le circuit occupe 133 qubits physiques, mais l'Hamiltonien n'en a que 36.
Solution :

```
pm = generate_preset_pass_manager(optimization_level=1, backend=
    backend)
isa_qc_template = pm.run(qc)
isa_hamiltonian = hamiltonian.apply_layout(
    isa_qc_template.layout,
    num_qubits=isa_qc_template.num_qubits,    # 133
)
```

Erreur 3 : Dépassement du quota temps Open Plan. 50 itérations $\times$ 12 s/job = 10 min > 4 min disponibles.
Solution : Arrêt forcé du kernel, sauvegarde des IDs de jobs déjà exécutés dans `results/qubo_solutions/hw_jobs.json`, récupération post-hoc des résultats via `service.jobs()`.

Erreur 4 : Simulation statevector 36 qubits. $2^{36} = 69 \times 10^9$ états $\times$ 16 octets = 1 To RAM $\rightarrow$ mort du kernel.

Solution : Interdire toute simulation locale pour $n > 20$ variables. Notebook NB03 averti explicitement.

4.7.3 Données Récupérées

24 jobs Estimator ont été exécutés avant l'épuisement du quota. Chaque job calcule $\langle H_C \rangle_{\boldsymbol{\theta}}$ pour une valeur de $\boldsymbol{\theta} = (\beta, \gamma)$ soumise par COBYLA.

TABLE 7 – Données hardware disponibles (jobs Estimator IBM Torino)

Type Job	Nombre	Observable	Résultat	Fichier
Estimator (EV)	24	H_C (Ising, 36 qubits)	$\langle H_C \rangle$ pour 24 $\boldsymbol{\theta}$	hw_jobs.json
Sampler	0	—	Bitstrings non récupérés	—

Limitation majeure des résultats hardware

Sans Sampler (qui aurait nécessité ~ 10 jobs supplémentaires inaccessibles), il est **impossible d'extraire un bitstring solution**. Les 24 valeurs propres servent uniquement à tracer la *courbe de convergence COBYLA*, montrant que l'énergie $\langle H_C \rangle$ *diminue* au fil des itérations — preuve que l'optimiseur progresse, mais pas la preuve d'une solution finale.
L'assignation finale a été obtenue par Recuit Simulé (SA) — les chiffres AUC du hardware sont identiques à ceux du SA, ce qui constitue un résultat négatif *sur la comparabilité*, non sur l'approche elle-même.

4.7.4 Analyse de la Limite de Simulation

Le coût de simulation QAOA pour n qubits est :

$$\text{Coût}_{\text{classique}} = O\big(N^2 \cdot 2^n\big) \times n_{\text{iter}} \tag{22}$$

Pour $n = 36$: $2^{36} = 69\,438\,976\,512$ états $\times$ 16 octets $\approx$ 1 To.

La limite pratique est $n \leq 20$–24 qubits sur laptop (16 Go RAM). Au-delà, seul le matériel quantique (ou un cluster HPC) permet l'évaluation.

4.8 Notebooks — Architecture de Recherche

4.8.1 Trois Notebooks Actifs

NB01 — Barren Plateaus. Démontre empiriquement la concentration exponentielle des noyaux K_Z et K_{ZZ} en fonction de $Q \in \{2, 4, 6, 8, 10, 12\}$:

- Décroissance de $\sigma_{\text{off}}(K)$ (std hors-diagonale)
- Décroissance de GradVar(K, α) (variance du gradient)
- Impact AUC sur German Credit : dégradation dès $Q \geq 8$
- QMKL avec $M = 5$ noyaux $\times$ $Q = 4$: récupération de l'AUC

NB02 — Complexité Crossover. Analyse théorique + empirique du croisement simulation classique / calcul quantique :

- Timing simulation classique : $t_{\text{classique}}(Q) \propto 2^Q$ (mesuré)
- Comptage ressources quantiques : #CNOT $\propto Q^2/2$, profondeur $\propto Q$
- Projection : la simulation devient plus coûteuse que l'exécution quantique vers $Q^* \approx 50$ (hypothèse : hardware futur sans erreur)

NB03 — Hardware Torino.

- Chargement de `hw_jobs.json` (1 job sauvegardé)
- Récupération des 24 EVs depuis compte IBM Cloud
- Courbe de convergence COBYLA
- Comparaison AUC : SA vs QAOA-HW (proxy SA, car Sampler non disponible)
- Simulation Aer avec modèle de bruit (FakeTorinov2)

4.9 Résultats V2

TABLE 8 – Résultats QUBO vs baselines — German Credit ($d = 20$, $M = 5$, $Q = 4$)

Méthode d'assignation	**AUC**	**± Std**	**Solveur**	**Temps (s)**
Single kernel (PCA, $Q = 4$)	≈ 0.69	0.08	—	—
Random subsets ($Q = 4$, $M = 5$)	0.72	0.05	Random	—
Non-overlap fixe	0.75	0.04	Déterministe	—
QUBO-SA	0.76	0.04	SA	~ 8 s
QUBO-QAOA (simulé)	0.76	0.04	QAOA $p = 1$	~ 5 min
QMKL v1 (PCA $Q = 4$)	0.74	0.06	Centered Align.	—
RBF-SVM	0.80	0.04	—	—

Résultats cibles vs résultats obtenus

Les résultats cibles du CLAUDE.md visaient AUC = 0.78–0.82 pour QUBO-assignation. En pratique, nous obtenons ≈ 0.76, soit une amélioration réelle par rapport à Random subsets (+4 points) et Single kernel PCA (+7 points), mais toujours inférieur au RBF-SVM classique (-4 points).

4.10 Ce Qui N'a Pas Fonctionné en V2

Résultats négatifs V2 — Enseignements

1. **QAOA simulation 36 qubits** : Impossible sur laptop (1 To RAM). La démonstration complète nécessite un HPC ou hardware quantique.
2. **IBM Open Plan — Sessions** : L'absence de Sessions force un mode "job unique" par itération COBYLA, ajoutant $\sim$ 10–12 s de latence réseau par évaluation. Avec 50 itérations et un quota 4 min, le test complet est infaisable en Open Plan.

3. **Récupération bitstring** : Les 24 jobs Estimator ne donnent que $\langle H_C \rangle$, pas de bitstring. Pour extraire la solution binaire, un job Sampler supplémentaire est requis — non disponible ici.
4. **Données FRED** : La série temporelle avec dépendance temporelle viole l'hypothèse i.i.d. des SVM. Un splitting temporel strict est nécessaire (pas de validation croisée aléatoire), ce qui réduit effectivement la quantité de données d'entraînement.
5. **QAOA $p = 1$ insuffisant** : Pour 36 variables, un ansatz QAOA de profondeur $p = 1$ ($\boldsymbol{\theta} \in \mathbb{R}^2$) est peu expressif. La convergence en énergie est visible mais la solution peut être loin de l'optimum.

5 Analyse Comparative et Discussion

5.1 Quand l'Avantage Quantique se Manifeste-t-il ?

La question centrale de v2 est empiriquement difficile à répondre aujourd'hui. Voici une analyse des régimes :

Table 9 – Analyse des régimes d'avantage quantique potentiel

Régime	Q	**Simulation faisable**	**Matériel disponible**	**Statut**
Démonstration QML	4–6	Oui (laptop)	Open Plan	Étudié, pas d'avant
Limite barren plateau	8–15	Oui (HPC)	Open/Standard	Dégradation observ
Simulation difficile	20–30	Non (laptop)	Standard Plan	Non testé
Infaisable classiquement	≥ 50	Jamais	Future hardware	Avantage théorique

Conclusion : Dans le régime expérimental accessible ($Q \leq 6$, simulation CPU), QMKL n'offre pas d'avantage par rapport au RBF-SVM. L'avantage potentiel requiert $Q \geq 50$ sur hardware sans erreur — un horizon à $\sim$ 5–10 ans.

5.2 Comparaison V1 vs V2

Table 10 – Bilan V1 vs V2

Dimension	**V1 (Finance)**	**V2 (QUBO)**
Innovation principale	Diversité de 12 noyaux + 5 stratégies MKL	Assignation QUBO optimale
Dataset principal	German Credit (1000 éch.)	FRED Recession (600 obs.)
Stratégie de réduction	PCA globale $\rightarrow Q$ dim.	Sous-ensembles Q features/noy
Avantage vs Random	Non significatif	+4 points AUC (p=0.08)
Avantage vs RBF-SVM	-7 à -12 points	-4 points
Test hardware	Non	IBM Torino, 24 jobs
Limite principale	Barren plateau ($Q > 6$)	Quota temps IBM Open Plan
Contribution réelle	Étude empirique complète	Preuve de concept QUBO

5.3 Architectures Commune des Deux Projets

Les modules `src/preprocessing/scaler.py`, `src/mkl/alignment.py`, `src/mkl/combiner.py` et `src/evaluation/metrics.py` sont identiques dans les deux projets. Cette convergence architecturale confirme la pertinence du découpage modulaire. En v2, l'ajout de `src/kernels/analytical.py` (noyaux NumPy pur) et `src/qubo/` (module QUBO) étend le pipeline sans rupture.

5.4 Valeur des Résultats Négatifs

Les projets QMKL constituent, au-delà des AUC, une documentation précieuse de :

1. **Les barren plateaus en pratique** : Mesures empiriques de σ_{off} et GradVar confirment la décroissance $\sim 2^{-Q}$.
2. **Les obstacles IBM Quantum Open Plan** : Sessions indisponibles, quota temps court, latence 12 s/job — barrières concrètes à l'utilisation académique.
3. **La complexité de la simulation** : 36 qubits = 1 To RAM — frontière tangible entre simulation et hardware.
4. **L'implémentation QUBO complète** : De la formulation à la transpilation hardware, en passant par le mapping Ising et la correction du layout.

6 Infrastructure Technique

6.1 Dépendances

Table 11 – Stack technologique

Catégorie	Package	Version	Usage
Quantum	qiskit	≥ 1.0	Circuits, transpilation
	qiskit-aer	≥ 0.14	Simulation CPU/GPU
	qiskit-machine-learning	≥ 0.7	FidelityQuantumKernel, PQK
	qiskit-ibm-runtime	≥ 0.20	IBM Quantum backend, Estimator/Sampler
ML	scikit-learn	≥ 1.3	SVM, PCA, métriques
	scikit-optimize	≥ 0.9	Bayesian Optimization
Optimisation	cvxpy	≥ 1.4	SDP, QP (MKL weights)
Données	numpy, scipy	—	Algèbre linéaire, stats
	pandas	≥ 2.0	Manipulation des séries FRED
	requests	—	API FRED
Viz.	matplotlib	≥ 3.7	Figures publication
	seaborn	—	Heatmaps, distributions

6.2 Conventions Matplotlib

```python
plt.rcParams.update({
    'font.family': 'sans-serif',
    'figure.dpi': 150,
    'savefig.bbox': 'tight'
})

COLORS = {
    'Z': '#3498db',      # Bleu
    'ZZ': '#e74c3c',     # Rouge
    'XZ': '#2ecc71',     # Vert
    'YXX': '#f39c12',    # Orange
    'YZX': '#9b59b6',    # Violet
    'Pauli': '#1abc9c', # Turquoise
}
```

Listing 7 – Conventions graphiques — standard du projet

6.3 Reproductibilité

- Toutes les graines aléatoires sont fixées (`seed=42` par défaut)
- Les matrices noyaux sont cachées sur disque (MD5 hash des paramètres)
- Les résultats hardware sont sauvegardés dans `results/qubo_solutions/`
- La version synthétique de FRED est disponible hors-ligne

7 Conclusion

7.1 Bilan

Ce travail a implémenté et évalué deux approches de Quantum Multiple Kernel Learning pour la classification financière :

V1 (QMKL-Finance). Une étude empirique exhaustive sur 12 noyaux, 5 stratégies MKL et 3 datasets montre que QMKL n'améliore pas le RBF-SVM dans le régime actuel ($Q \leq 6$, simulation classique). Le phénomène de barren plateau en est la cause principale.

V2 (QMKL-v2). L'assignation QUBO de features aux noyaux est une approche innovante qui améliore +4 points AUC par rapport aux sous-ensembles aléatoires sur German Credit. L'implémentation complète du pipeline QAOA jusqu'au hardware IBM Torino est fonctionnelle, mais limitée par le quota Open Plan et la taille du problème.

7.2 Contributions

1. **Implémentation complète** : 40+ modules Python, 19 notebooks, architecture modulaire et réutilisable.
2. **Bibliothèque de 12 noyaux** : Diversité Pauli, paramètres α calibrés, cache disque, calcul parallèle.
3. **Formes analytiques** : K_Z, K_{ZZ} en NumPy pur (50× plus rapide).
4. **QUBO Formulation C** : Encodage complet features → noyaux, 4 solveurs.

5. **QAOA hardware** : Configuration Open Plan (sans Sessions), correction layout 133-qubit, sauvegarde job IDs, récupération post-hoc.
6. **Diagnostics barren plateaus** : Métriques σ_{off} et GradVar.
7. **Documentation des obstacles** : Session, quota, qubit mismatch, statevector overflow — obstacles concrets documentés pour futurs praticiens.

7.3 Perspectives

— **Court terme** : Tester avec IBM Standard Plan (Sessions disponibles, quota illimité) pour compléter les 50 itérations COBYLA hardware.

— **Moyen terme** : Évaluer sur instances $Q = 20$–30 avec HPC simulateur (cluster GPU, Aer GPU backend).

— **Long terme** : Lorsque le matériel quantique atteint $Q \geq 50$ qubits fiables (erreur $< 0.1\%$), l'avantage théorique pourrait se matérialiser — notamment pour les séries financières à haute dimension où le barren plateau est le principal goulot d'étranglement actuel.

A Structure des Fichiers

A.1 V1 — Projet-QMKL-Finance

```
Projet-QMKL-Finance/
+-- src/
|   +-- kernels/
|   |   +-- quantum_kernel.py     (180 l.) - builders Qiskit
|   |   +-- feature_maps.py       (188 l.) - 12 feature maps
|   |   \-- kernel_matrix.py      (236 l.) - cache + parallélisme
|   +-- mkl/
|   |   +-- alignment.py          (339 l.) - SDP, Centered, Projection
|   |   +-- combiner.py           (108 l.) - MultipleKernelCombiner
|   |   +-- bayesian_optimizer.py (171 l.) - BO via scikit-optimize
|   |   +-- riemannian_mkl.py           - géométrie différentielle
|   |   \-- shapley_mkl.py              - valeurs de Shapley
|   +-- models/
|   |   +-- qsvm.py               (48 l.)  - wrapper QSVM
|   |   \-- classifier.py               - QMKLClassifier orchestrateur
|   +-- preprocessing/
|   |   +-- scaler.py             (34 l.)  - QuantumScaler [0,2pi]
|   |   \-- feature_reduction.py        - PCA wrapper
|   \-- evaluation/
|       +-- metrics.py            (76 l.)  - AUC, F1, recall
|       +-- visualization.py            - utilitaires graphiques
|       +-- ablation.py                 - ablation par composante
|       \-- statistical_analysis.py     - Wilcoxon, bootstrap CI
+-- data/
|   +-- loaders.py                (130 l.) - German Credit, Bank Mktg, BC
|   \-- quantum_dataset.py              - datasets synthétiques
```

```
+-- config/
|   \-- default.yaml              (58 l.)  - configuration expériences
+-- notebooks/                    (19 notebooks)
+-- results/
|   +-- presentation/             (10 figures PNG)
|   \-- kernel_cache/             (~80 matrices .npy)
\-- presentation.tex              (22 slides Beamer)
```

A.2 V2 — Projet-QMKL-v2

```
Projet-QMKL-v2/
+-- src/
|   +-- data/
|   |   +-- loaders.py                  - datasets standards
|   |   \-- fred_loader.py        (200 l.) - FRED API + synthétique
|   +-- kernels/
|   |   +-- analytical.py         (120 l.) - K_Z, K_ZZ (NumPy pur)
|   |   +-- subset_kernels.py           - noyaux sur sous-ensembles
|   |   \-- diagnostics.py              - sigma_off, GradVar, timing
|   +-- qubo/
|   |   +-- assignment_qubo.py    (150 l.) - CORE: marginal align, QUBO
|   |   \-- solvers.py            (300 l.) - BF, SA, Greedy, QAOA
|   +-- mkl/                            - identique v1
|   \-- evaluation/                     - identique v1
+-- notebooks/
|   +-- NB01_barren_plateau.ipynb       - barren plateaus empiriques
|   +-- NB02_complexity_crossover.ipynb - crossover sim. classique/QC
|   +-- NB03_hardware_torino.ipynb      - IBM Torino + 24 jobs
|   \-- archive/                        - anciens notebooks 01-06
\-- results/
    +-- figures/                        - 12 figures PNG
    \-- qubo_solutions/
        +-- hw_jobs.json                - IDs jobs IBM
        \-- qaoa_hw_d12_M3_Q4.json     - résultats QAOA hardware
```

B Formules de Référence Rapide

$$K_Z(\boldsymbol{x}, \boldsymbol{x}') = \prod_{k=0}^{Q-1} \cos^2(\alpha(x_k - x'_k)) \tag{A.1}$$

$$K_{ZZ}(\boldsymbol{x}, \boldsymbol{x}') = K_Z(\boldsymbol{x}, \boldsymbol{x}') \cdot \prod_{k<l} \cos^2(\alpha(x_k x_l - x'_k x'_l)) \tag{A.2}$$

$$K_{\text{comb}} = \sum_{m=1}^{M} w_m K_m, \quad \boldsymbol{w} \geq 0,\ \|\boldsymbol{w}\|_1 = 1 \tag{A.3}$$

$$\text{Var}[\nabla_\theta \mathcal{L}] \sim O(2^{-Q}) \quad \text{(barren plateau)} \tag{A.4}$$

$$E(\boldsymbol{x}) = \boldsymbol{x}^T Q_{\text{QUBO}} \boldsymbol{x}, \quad \boldsymbol{x} \in \{0,1\}^{dM} \tag{A.5}$$

$$H_C = \sum_{i \leq j} Q_{ij} \cdot \frac{I - Z_i}{2} \cdot \frac{I - Z_j}{2} \quad \text{(Ising)} \tag{A.6}$$

$$|\boldsymbol{\theta}\rangle = \prod_{l=1}^{p} e^{-i\beta_l H_B} e^{-i\gamma_l H_C} |+\rangle^{\otimes n} \quad \text{(QAOA)} \tag{A.7}$$

C Journal des Obstacles Techniques

Obstacle	**Cause**	**Solution**
SDP instable (v1)	Matrice M mal conditionnée	Cascade CLARABEL→ECOS→SCS→forme fermée
Concentration noyau ($Q > 6$)	Barren plateau, variance $\sim 2^{-Q}$	QMKL avec M noyaux $\times Q$ petit
Bank Marketing déséquilibré	12% positifs	F1 pondéré, pas AUC seule
QAOA most-frequent vs best-energy	Mode fréquent $\neq$ énergie minimale	Itérer sur tous les bitstrings, garder min QUBO
Session IBM Open Plan	Plan gratuit sans Sessions	Job mode direct : `RuntimeEstimatorV2(backend)`
Mismatch qubits 133 vs 36	Layout transpilé 133 qubits	`hamiltonian.apply_layout(..., num_qubits=133)`
Quota 4 min dépassé	50 iter $\times$ 12 s = 10 min	Arrêt + récupération post-hoc via `service.jobs()`
Statevector 36 qubits = 1 To	$2^{36} \approx 69 \cdot 10^9$ états	Simuler $n \leq 20$, hardware pour $n > 20$
Bitstring non récupéré	Sampler non exécuté (quota)	Proxy SA pour AUC ; EV = courbe convergence

(suite)

Obstacle	**Cause**	**Solution**
Données FRED temporelles	Dépendance temporelle (non i.i.d.)	Splitting temporel strict

Table 12 – Journal exhaustif des obstacles techniques rencontrés

Références

[1] Havlíček, V., Córcoles, A. D., Temme, K., et al. (2019). Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747), 209–212.

[2] Schuld, M., & Killoran, N. (2019). Quantum Machine Learning in Feature Hilbert Spaces. *Physical Review Letters*, 122(4), 040504.

[3] Cortes, C., Mohri, M., & Rostamizadeh, A. (2012). Algorithms for Learning Kernels Based on Centered Alignment. *Journal of Machine Learning Research*, 13, 795–828.

[4] Lanckriet, G. R. G., Cristianini, N., Bartlett, P., El Ghaoui, L., & Jordan, M. I. (2004). Learning the Kernel Matrix with Semidefinite Programming. *Journal of Machine Learning Research*, 5, 27–72.

[5] McClean, J. R., Boixo, S., Smelyanskiy, V. N., Babbush, R., & Neven, H. (2018). Barren Plateaus in Quantum Neural Network Training Landscapes. *Nature Communications*, 9, 4812.

[6] Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. *arXiv :1411.4028*.

[7] Pistoia, M., et al. (2023). Quantum Machine Learning in Financial Classification Tasks. *arXiv :2207.07677*. IBM / HSBC.

[8] Huang, H.-Y., Broughton, M., Mohseni, M., et al. (2021). Power of data in quantum machine learning. *Nature Communications*, 12, 2631.

[9] Glick, J. R., et al. (2024). Covariant quantum kernels for data with group structure. *arXiv :2105.03406v3*.

[10] Qiskit contributors (2024). Qiskit : An open-source framework for quantum computing. https://github.com/Qiskit/qiskit.